

Introduction

For a Computer Professional, there are both practical and theoretical reasons to study algorithms. One such practical reason is to know a standard set of important algorithms for different areas of computing, in addition one should be able to design new algorithms and analyze their efficiency.

Note :- Study of Algorithms is called "Algorithmics"

Another reason for studying algorithmics is their usefulness in developing analytical skills. After all, algorithms can be seen as special kind of solutions to problems.

What are Algorithms?

Algorithms are a sequence of unambiguous instructions for solving a problem i.e. for obtaining a required output for any legitimate input in a finite amount of time.

One important note is that an algorithm is not the whole program or code of the problem. It is simply a logic to be implemented to solve the problem. It can be described as a pseudo-code.

Characteristics of Algorithm:-

- 1) Well defined Inputs.
- 2) Well defined Outputs
- 3) Unambiguity.
- 4) Finiteness
- 5) Language Independent
- 6) Effectiveness & Feasibility

Performance Analysis of Algorithms

Sonal Saxena
Unit 1

19

When there are multiple alternative algorithms to solve a problem, we analyze them and pick the one which is best suitable for our requirements.

Performance of an algorithm is a process of making evaluative judgement about algorithms.

Generally, the performance of an algorithm depends on the following elements:-

- 1.) Whether the algo. is providing the exact solution of the problem?
- 2.) Whether it is easy to understand?
- 3.) Whether it is easily implemented?
- 4.) How much memory (space) it is grabbing?
- 5.) How much time it is taking to solve a problem?

But when we practically analyse any algorithm, we ignore all the above elements except space and time of the algorithm.

So, performance analysis of an algorithm is performed by using following measures:-

- i) Space Complexity :- space required to complete the task of an algorithm.
- ii) Time Complexity :- time required to execute a task/algorithm.

Asymptotic Notations

Unit I Sonal Saxena, (3)

Whenever we want to perform analysis of algorithm, we need to calculate the complexity of that algorithm. But complexity can't provide exact amount of resources required. So for this, we represent the complexity in a general form called Notations, which produces the basic nature of algorithm.

So,
Asymptotic notation is a mathematical representation of its complexity.

In this notation, we only consider the most significant term. For ex:-

Algo 1 :- $5n^2 + 10n + 2$

Algo 2 :- $10n^2 + 8n + 3$

In Algo 1, $10n + 2$ is least significant in comparison to $5n^2$, so $5n^2$ is most significant. Similarly, in algo 2, $8n + 3$ is least significant and $10n^2$ is most significant. So in both the algorithms, we can ignore least significant terms.

There are majorly 3 types of asymptotic notations:-

- 1.) Big Oh (O)
- 2.) Omega (Ω)
- 3.) Theta (Θ)

Let us now understand each of them one by one.

Big Oh (O) Notation

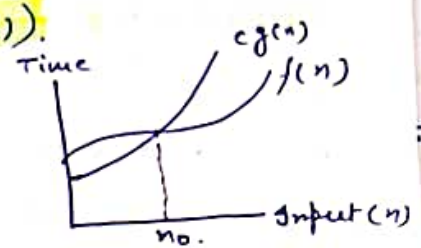
It is used to define the upper bound of an algorithm in terms of time complexity. This means Big Oh notation always defines the maximum time taken by any algorithm for all input values. It describes the worst case of any algorithm.

Consider a function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term.

If $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$, $c > 0$ and $n_0 \geq 1$.

Then we can represent $f(n)$ as $O(g(n))$.

$$f(n) = O(g(n))$$

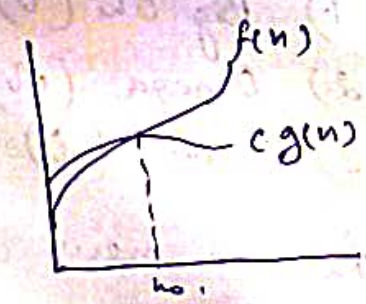


Omega (Ω) Notation

It is used to define lower bound of an algorithm in terms of time complexity. It always indicates the minimum time required by an algorithm for all input values. It describes the best case of an algorithm time complexity.

If $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$, $c > 0$ and $n_0 \geq 1$, then we can represent $f(n)$ as $\Omega(g(n))$.

$$f(n) = \Omega(g(n))$$



Theta (Θ) Notation

It is used to define average bound of an algorithm in terms of Time complexity. It always indicates the average time required by any algorithm for all input values.

Unit I Sonal Saxena (5)
that means, theta notation describes the average case of an algorithm time complexity.
Consider a function $f(n)$ and $g(n)$. If $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, $c_1 > 0$, $c_2 > 0$ and $n_0 \geq 1$. Then we can represent $f(n)$ as $\Theta(g(n))$.

$$f(n) = \Theta(g(n))$$

How to Calculate Complexity?

1) Step Count Method in Algorithm.

In this method, we count the no. of times one instruction is executing. Some assumptions in this method are:-

- There is no count for $\{$ or $\}$.
- Each basic statement like "assignment" and "return" have a count of 1.
- If a basic statement is iterated, then multiply by the no. of times the loop is run.
- The loop statement is iterated n times, it has a count of $(n+1)$. Here the loop runs n times for the true case and a check is performed for the loop exit (the false condition), hence the additional 1 in the count.

Dr.

Sonal Saxena
Unit I

Recurrence Relations

It is an equation or inequality that describes a function in terms of its values on smaller inputs. To solve a recurrence relation means to obtain a function defined on natural numbers that satisfy the recurrence.

Let us first understand recurrence relation with an example of Binary Search first.

Consider a list of numbers (sorted) for searching:-

index $\rightarrow$	1	2	3	4	5	6	7	8	9
	11	12	13	14	15	16	17	18	19
	$\leftarrow i$								$\rightarrow j$

Now in general, "Recurrence" means anything which is happening or occurring repeatedly. Here, we will find a function which is repeating again and again. Let us write general algo for Binary Search:-

BS(a, i, j, x), where a = array, i = lower index, j = higher index and x = no. to be searched.

if $a[mid] == x$

"No." is found "return(mid)"

else if $a[mid] > x$

set BS($a, i, mid-1, x$)

else

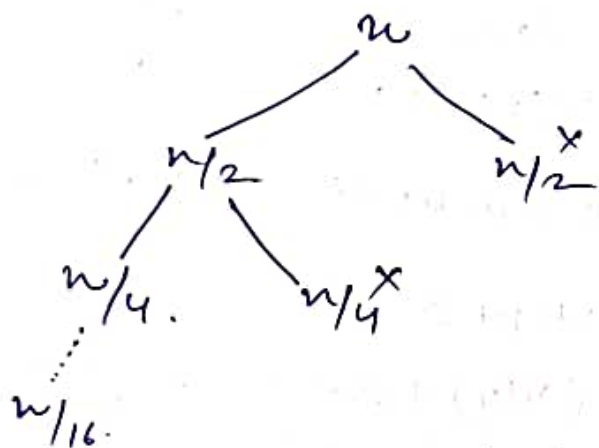
set BS($a, i, mid+1, x$)

Now in summarised form, whenever we search an element in a list using Binary Search, we first compare it with middle element. If the no. is found at the middle element, then its complexity becomes constant, i.e. $O(1)$. But assume if it is greater / lesser than the middle element, we have to move in that particular half of the list. The other half of the list is of no use to us. And this

Howish

Gr
some procedure is repeated till either we find the no. or the no. doesn't exist in the list.

Unit I
Soal Savana (7)
So, now we can conclude that, if we have total 'n' numbers, i.e., we always divide it into 2 using mid element. which becomes $(n/2)$. Again if we repeat the procedure, this $(n/2)$ list is further divided to $(n/4)$ and then to $(n/8)$ etc.....



So, now the recurrence relation for this becomes -

$$T(n) = T(n/2) + c$$

This c is some constant amount of time which can be the time taken to compare the middle value with the no. to be searched etc.

Now, we will understand how to solve recurrence relation using Substitution Method.

Unit I

Substitution Method

It is also called Back Substitution Method. Let us take the same Binary Search recurrence relation which we devised earlier:-

$$T(n) = \begin{cases} T(n/2) + c & \text{if } n > 1 \\ 1 & \text{if } n = 1 \end{cases}$$

$$T(n) = T(n/2) + c \quad \text{--- (1)}$$

$$T(n/2) = T(n/4) + c \quad \text{--- (2)}$$

$$T(n/4) = T(n/8) + c \quad \text{--- (3)}$$

Now let us back substitute $T(n/2)$ in equation 1

$$T(n) = T(n/2) + c$$

after substitution:- $= T(n/4) + c + c \quad \text{--- (4)}$

$$= T(n/2^2) + 2c$$

Now as we have $T(n/4)$ in equation (4) we can back substitute the value of $T(n/4)$ in it. So after substitution it becomes:-

$$= (T(n/8) + c) + 2c$$

$$\Rightarrow T(n/2^3) + 3c$$

$$\vdots$$

$$\Rightarrow T(n/2^k) + kc$$

Now if I want to make " $n/2^k$ " this whole term as 1, I need to put $n = 2^k$.

$$\Rightarrow T\left(\frac{n}{n}\right) + kc$$

$$\Rightarrow T(1) + kc$$

$$\Rightarrow 1 + kc \quad \text{--- (5)}$$

But we took complexity in terms of n , so we'll consider

the term $n = 2^k$. Take log both the side, (9)

$$\log n = \log 2^k$$

$$\log n \Rightarrow k \log 2$$

as $\log 2 = 1$, so $k = \log n$

Now put the value of $k = \log n$ in equation 5 which is

$$\Rightarrow 1 + k c = 1 + \log n c \quad \text{--- (6)}$$

Here in (6), 1 and c both are constant, so it can be written as $O(\log n)$, which is the complexity of Binary Search.

Example 1:-

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ n * T(n-1) & \text{if } n > 1 \end{cases}$$

Solution:-

$$T(n) = n * T(n-1) \quad \text{--- (1)}$$

Now substituting $(n-1)$ in place of "n".

$$\begin{aligned} T(n-1) &= (n-1) * T((n-1)-1) \\ &= (n-1) * T(n-2) \quad \text{--- (2)} \end{aligned}$$

Now putting $(n-2)$ instead of n,

$$\begin{aligned} T(n-2) &= (n-2) * T((n-2)-1) \\ &= (n-2) * T(n-3) \quad \text{--- (3)} \end{aligned}$$

Now back substitute eq (2) in (1) :-

$$\begin{aligned} T(n) &= n * T(n-1) \\ T(n) &= n * (n-1) * T(n-2) \quad \text{--- (4)} \end{aligned}$$

Again back substituting eq (3) in (4)

$$T(n) = n * (n-1) * (n-2) * T(n-3)$$

So, in generalised form, it will,

$$T(n) = n * (n-1) * (n-2) * (n-3) * \dots * T(n-(n-1))$$

it will run till (n-1) steps.

the term

$$n = 2^k$$

Take log both the side,

$$\log n = \log 2^k$$

$$\log n \Rightarrow k \log 2$$

as $\log 2 = 1$, so $k = \log n$

Now put the value of $k = \log n$ in equation 5 which is

$$1 + kc$$

$$\Rightarrow 1 + \log n c \quad \text{--- (6)}$$

Here in (6), 1 and c both are constant, so it can be written as $O(\log n)$, which is the complexity of Binary Search.

Example 1:-

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ n * T(n-1) & \text{if } n > 1 \end{cases}$$

Solution:-

$$T(n) = n * T(n-1) \quad \text{--- (1)}$$

Now substituting $(n-1)$ in place of "n".

$$\begin{aligned} T(n-1) &= (n-1) * T(n-1-1) \\ &= (n-1) * T(n-2) \quad \text{--- (2)} \end{aligned}$$

Now putting $(n-2)$ instead of n,

$$\begin{aligned} T(n-2) &= (n-2) * T(n-2-1) \\ &= (n-2) * T(n-3) \quad \text{--- (3)} \end{aligned}$$

Now back substitute eq (2) in (1) :-

$$\begin{aligned} T(n) &= n * T(n-1) \\ T(n) &= n * (n-1) * T(n-2) \quad \text{--- (4)} \end{aligned}$$

Again back substituting eq (3) in (4)

$$T(n) = n * (n-1) * (n-2) * T(n-3)$$

So, in generalised form, it will,

$$T(n) = n * (n-1) * (n-2) * \dots * T(n-(n-1))$$

it will run till $(n-1)$ steps.

So, this equation becomes,

$$T(n) = n * (n-1) * (n-2) * (n-3) \dots * (n-(n-1))$$

$$\Rightarrow T(n) = n * (n-1) * (n-2) * (n-3) \dots * (n-(n+1))$$

$$= n * (n-1) * (n-2) * (n-3) \dots * (1)$$

$$= n * (n-1) * (n-2) * (n-3) \dots 3 * 2 * 1$$

Taking "n" common from the above series,

$$= n * n(1 - \frac{1}{n}) * n(1 - \frac{2}{n}) * n(1 - \frac{3}{n}) * \dots * n(\frac{2}{n}) * n(\frac{1}{n})$$

$$= O(n^n) \quad \{ \text{factorial type complexity} \}$$

Example 2:- $T(n) = 2T(\frac{n}{2}) + n$ — (1) $T(1) = 1$

Sol Now as we can see in eq 1, n is converting to n/2 on the RHS. So, we have guess the solution that how can we proceed with recurrence relation. So now we will put n/2 in place of n in eq (1), which will be:-

$$T(n) = 2T(\frac{n}{2}) + n \quad \text{--- (1)}$$

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + \frac{n}{2} \quad \text{--- (2)}$$

Now back substituting eq (2) in eq (1);

$$T(n) = 2 \{ 2T(\frac{n}{4}) + \frac{n}{2} \} + n$$

$$= 4T(\frac{n}{4}) + 2n + n$$

$$= 4T(\frac{n}{4}) + 3n$$

$$= 2^2 T(\frac{n}{2^2}) + 2n \quad \text{--- (3)}$$

Again substituting n/4 in n eq (1):-

$$T(\frac{n}{4}) = 2T(\frac{n}{8}) + \frac{n}{4} \quad \text{--- (4)}$$

Back substituting eq (4) in eq (3)

$$\begin{aligned}
 T(n) &= 4 \left(2T\left(\frac{n}{2}\right) + \frac{n}{4} \right) + 2n \\
 &= 8T\left(\frac{n}{2}\right) + n + 2n \\
 &= 2^3 T\left(\frac{n}{2^3}\right) + 3n
 \end{aligned}$$

So the series becomes,

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + i \cdot n$$

$$\text{Let } \frac{n}{2^i} = 1 \Rightarrow n = 2^i$$

Taking log both sides,

$$\log n = \log 2^i$$

$$i = \log_2 n$$

So general equation becomes,

$$T(n) = n T(1) + n \log_2 n$$

In finding time complexity we always ignore least significant term, so it becomes

$$O(n \log n)$$

Master Theorem for Recurrence Relations Unit I

Earlier we have seen substitution method for solving recurrence relation which can be applied on any recurrence relation without any constraint but it is a bit slow method as we need to guess the solution in it.

So, Master Theorem is considered to be a faster method as compared to substitution method but it has some constraints and it can't be applied on any recurrence relation.

So, let's see what kind of recurrence relation should be there for applying Master Theorem.

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$
$$a \geq 1, b > 1$$

Condition for Master Theorem

Now see the below 3 equations :-

- i) $T(n) = T(n-1) + 1$
- ii) $T(n) = 8T\left(\frac{n}{2}\right) + n^2$
- iii) $T(n) = T\left(\frac{n}{2}\right) + c$

In eq i), value of $a=1$ and $b=1$, which isn't feasible for Master Theorem. So we can't apply Master Theorem on this.

In eq ii) value of $a=8$ and $b=2$ which is satisfying the condition.

In eq iii) value of $a=1$ and $b=2$ which again satisfies Master Theorem constraint.

coming back to the solution, if recurrence relation satisfies constraint of Master Theorem, then the solution for Master Theorem is fixed:-

Solution :- $T(n) = n^{\log_b a} * [U(n)]$,

Where value of n, a, b is already known, but only $U(n)$ needs to be known which actually depends on ~~form~~ $h(n)$ in Recurrence Relation.

$$h(n) = \frac{f(n)}{n^{\log_b a}}$$

Relation between $h(n)$ and $U(n)$ is:-

if $h(n)$	$U(n)$
1) $n^x, x > 0$	$O(n^x)$
2) $n^x, x < 0$	$O(1)$
3) $(\log n)^i, i \geq 0$	$\frac{(\log_2 n)^{i+1}}{i+1}$

Example:- $T(n) = 8 + \left(\frac{n}{2}\right) + n^2$

Solution :- Here, $a = 8, b = 2$ and $f(n) = n^2$
Now its solution would be :-

$$T(n) = n^{\log_b a} * U(n)$$

$$= n^{\log_2 8} * U(n)$$

$$= n^3 U(n)$$

$$[\because \log_a b = x \text{ then } a^x = b]$$

Now $U(n)$ depends on $h(n)$, So $h(n) =$

$$\frac{f(n)}{n^{\log_b a}} = \frac{n^2}{n^{\log_2 8}} = \frac{n^2}{n^3} = \frac{1}{n} = n^{-1}$$

Now see the relationship table of $h(n)$ $U(n)$, where case 2 is

sonal saxa unit I

Unit I

applicable. So coming back to eq (1),

$$\begin{aligned} &= n^3 \cdot 1 \\ &= O(n^3) \end{aligned}$$

Example 2 :- $T(n) = T(n/2) + C$

solution = $a=1, b=2, f(n)=C.$

Now solution of M.T is :-

$$\begin{aligned} T(n) &= n^{\log_b a} U(n) \\ &= n^{\log_2 1} U(n) \\ &= n^0 U(n) \\ &= 1 * U(n) \\ &= U(n) \end{aligned}$$

$$[\therefore \log_2 1 = 0]$$

$$[x^0 = 1]$$

Now for finding $U(n)$, we need to calculate $h(n)$

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{c}{n^{\log_2 1}} = \frac{c}{n^0} = \frac{c}{1} = c$$

Now see the relationship table, case 1 and 2 aren't applicable. So we need to make it case 3. So case

3 is $(\log n)^i$ if $i \geq 0 = \frac{(\log_2 n)^{i+1}}{i+1} = \frac{(\log_2 n)^{0+1}}{0+1} = \log_2 n$
 $= O(\log_2 n)$

Divide & ConquerUnit I

It is one of the design approach used for solving large problems. by :-

- i) breaking the problem into smaller sub-problems (divide)
- ii) solving the sub-problems (Conquer)
- iii) Combining them to get the desired output. (Combine)

To solve a problem using divide & conquer strategy, usually recursion is used. There are various algorithms which are based on this approach. Some famous DnC algorithms are:-

- i) Binary Search
- ii) Merge Sort
- iii) Quick Sort.
- iv) Min-Max

Min Max Algorithm

Using Min Max Algorithm, we can find the smallest and the largest element of a given list. This process can be done using 2 approaches, one is linear, which is time consuming and another one is DnC which is considered to be efficient. So, here we will see DnC approach for solving min max problem.

Let us find the Min & Max of the below given list:-

2 17 9 8 5 14 3 6 1 18

Step 1: There are total 10 elements in above list. Start indexing them from 1 to 10.

1	2	3	4	5	6	7	8	9	10
2	17	9	8	5	14	3	6	1	18

= unit I

Step 2: Now take two variables i and j and assign them lower index and higher index respectively. So,
 $i = 1$ and $j = 10$.

Step 3:- As we are using DMC, we need to break the list into smaller lists. For this, we will find out middle element index and break the list from there. For finding mid do:-

$$\text{Mid} = \frac{i+j}{2} = \frac{1+10}{2} = \frac{11}{2} = 5.5$$

Take the lower value out of 5.5 $\Rightarrow 5$. So index 5 is the middle node and we will divide the list from there.

list 1

2 17 9 8 5

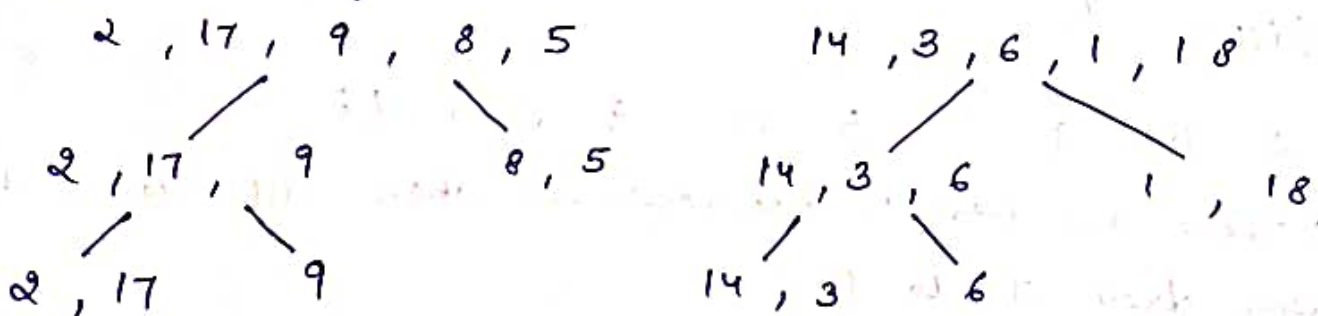
list 2

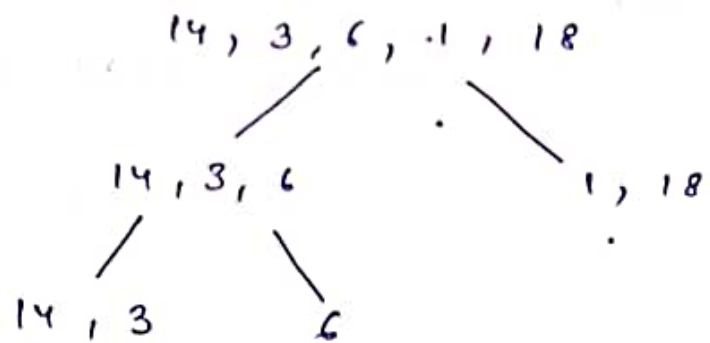
14 3 6 1 18

Again divide both the lists using same step 3.



Again apply same procedure.





$$\begin{array}{ll} \min = 3 & \min = 6 \\ \max = 14 & \max = 6 \end{array}$$

$$\left\{ \begin{array}{l} \min = 3 \\ \max = 14 \end{array} \right\}$$

$$\begin{array}{ll} \min = 3 & \min = 1 \\ \max = 14 & \max = 18 \end{array}$$

$$\left\{ \begin{array}{l} \min = 1 \\ \max = 18 \end{array} \right\}$$

$$\left\{ \begin{array}{l} \min = 1 \\ \max = 18 \end{array} \right\}$$

11

$$T(n) = T(n/2)$$

and because we are having 2 comparisons on every level, one for max and one for min so, it is

Unit I

social sciences

(12)

taking 2 units of constant time, so we can present it like -

$$T(n) = 2T(n/2) + 2$$

Total no. of comparisons done using D & C approach:-

$$= \frac{3n}{2} - 2$$

If you use linear approach it will be :-

$$2n - 2$$

which is very high in comparison to D & C approach.